

```

import { initializeApp } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-app.js";
import { getAuth, onAuthStateChanged } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";
import { getDatabase, ref, set, onValue, remove, get, child, update } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-database.js";

const firebaseConfig = {
  apiKey: "AIzaSyB0gShB0u05UszCBLS-bpTl2f3AI7_I-pY",
  authDomain: "reservasisd.firebaseio.com",
  databaseURL: "https://reservasisd-default-rtdb.firebaseio.com",
  projectId: "reservasisd",
  storageBucket: "reservasisd.firebaseio.com",
  messagingSenderId: "637702189208",
  appId: "1:637702189208:web:49ff477b35e299564ca0ed"
};

const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
const db = getDatabase(app);
const configHoraReporte = document.getElementById('config-hora-reporte');
const btnGuardarHora = document.getElementById('btn-guardar-hora');
const txtStatusConfig = document.getElementById('txt-status-config');
const btnBorrarIntentos = document.getElementById('btn-borrar-intentos');
//const selectIntentos = document.getElementById('select-intentos-acceso');

// =====
// CLAVE DE API DE IMGBB
// =====
const IMGBB_API_KEY = "85fceaf48792a118d9543ddc94c116ab";

// ELEMENTOS DEL DOM
const formNuevo = document.getElementById('form-nuevo-recurso');
const listaInventario = document.getElementById('lista-inventario');
const btnVolver = document.getElementById('btn-volver');

const selectIntentos = document.getElementById('select-intentos-acceso');
//const formNuevoProfesor = document.getElementById('form-nuevo-profesor');
const inputNombre = document.getElementById('profe-nombre');
const inputEmail = document.getElementById('profe-email');

// NUEVOS ELEMENTOS DEL DOM PARA ADMINS
const formNuevoAdmin = document.getElementById('form-nuevo-admin');
const listaAdministradores = document.getElementById('lista-administradores');

// NUEVOS ELEMENTOS DEL DOM PARA PROFESORES
const formNuevoProfesor = document.getElementById('form-nuevo-profesor');

```

```

const listaProfesores = document.getElementById('lista-profesores');

// 1. GUARDIAN Estricto Dinámico
onAuthStateChanged(auth, async (user) => {
  if (!user) {
    window.location.href = "index.html";
    return;
  }
  const email = user.email;
  // EXCEPCIÓN DIRECTA:
  if (email === "laggerro2@gmail.com") {
    inicializarPanel();
    return;
  }
  // Para cualquier otro correo, verificamos en la RTDB
  try {
    const emailLimpio = email.replace(/\.\/g, '_');
    const dbRef = ref(db);
    const snapshotAdmin = await get(child(dbRef,
`administradores/${emailLimpio}`));
    if (snapshotAdmin.exists()) {
      inicializarPanel();
    } else {
      alert("Acceso denegado. Se requieren credenciales de administrador.");
      window.location.href = "equipos.html";
    }
  } catch (error) {
    console.error("Error al verificar permisos de administrador:", error);
    window.location.href = "equipos.html";
  }
});

// Modificamos la función inicializadora para cargar todo
function inicializarPanel() {
  escucharYListarEquipos();
  escucharYListarAdmins();
  escucharYListarProfesores();
  escucharYListarIntentosAcceso(); // <-- AGREGAR ESTA LÍNEA
  cargarConfiguracionHora();
}

// =====
// MÓDULO A: CONTROL DE INVENTARIO (EQUIPOS)
// =====

// ENVIAR FORMULARIO (SUBIDA A IMGBB + REGISTRO FIREBASE)
formNuevo.addEventListener('submit', async (e) => {
  e.preventDefault();
  const categoria = document.getElementById('recurso-categoria').value;
  const nombre = document.getElementById('recurso-nombre').value.trim();

```

```

    const cantidad = parseInt(document.getElementById('recurso-cantidad').value);
    const descripcion =
document.getElementById('recurso-descripcion').value.trim();
    const inputImagen = document.getElementById('recurso-imagen').files[0];

    const idRecurso = nombre.toLowerCase()
        .replace(/[^a-z0-9]+/g, '_')
        .replace(/(^_|_$/g, '');

    let urlFinalImagen = "https://i.ibb.co/5z0RJbk/default-recurso.png"; // Imagen
por defecto por seguridad

    try {
        // 1. Subida a ImgBB en lugar del PHP local
        if (inputImagen) {
            console.log("Subiendo imagen a ImgBB");
            const formData = new FormData();
            formData.append('image', inputImagen);

            const responseImgBB = await
fetch(`https://api.imgbb.com/1/upload?key=${IMGBB_API_KEY}`, {
                method: 'POST',
                body: formData
            });

            const dataImgBB = await responseImgBB.json();

            if (!dataImgBB.success) {
                throw new Error("No se pudo subir la imagen al servidor de
ImgBB.");
            }

            // Guardamos la URL pública directa que nos da ImgBB
            urlFinalImagen = dataImgBB.data.url;
            console.log("Imagen subida con éxito: ", urlFinalImagen);
        }

        // 2. Registro directo en Firebase RTDB
        const recursoRef = ref(db, `inventario/${categoria}/${idRecurso}`);
        await set(recursoRef, {
            nombre: nombre,
            stock_total: cantidad,
            descripcion: descripcion,
            imagen: urlFinalImagen
        });

        alert("¡Equipo agregado con éxito al inventario!");
        formNuevo.reset();
    } catch (error) {
        console.error("Error al registrar:", error);
    }

```

```

        alert(`Falló el registro: ${error.message}`);
    }
});

// LISTAR EQUIPOS EN AJUSTES.JS
function escucharYListarEquipos() {
    const inventarioRef = ref(db, 'inventario');
    onValue(inventarioRef, (snapshot) => {
        listaInventario.innerHTML = '';
        const categorias = snapshot.val();
        if (!categorias) {
            listaInventario.innerHTML = '<p class="text-center text-gray-400 py-4">No hay equipos en el inventario.</p>';
            return;
        }
        Object.keys(categorias).forEach(idCategoria => {
            const recursos = categorias[idCategoria];
            Object.keys(recursos).forEach(idIdRecurso => {
                const item = recursos[idIdRecurso];
                const itemHTML = `
                    <div class="flex items-center justify-between p-3 bg-gray-50 border rounded-xl hover:shadow-sm transition-all mb-2">
                        <div class="flex items-center gap-3">
                            
                            <div>
                                <h4 class="font-bold text-gray-800 text-sm leading-tight">${item.nombre}</h4>
                                <p class="text-xs text-gray-500 uppercase tracking-wide font-semibold">${idCategoria}</p>
                            </div>
                        </div>

                        <!-- Control de Stock Interactivo -->
                        <div class="flex items-center gap-2">
                            <span class="text-xs text-gray-500 font-bold">Stock:</span>
                            <div class="flex items-center bg-white border rounded-lg overflow-hidden shadow-sm">
                                <button onclick="cambiarStock('${idCategoria}', '${idIdRecurso}', ${item.stock_total}, -1)" class="px-2 py-1 bg-gray-100 hover:bg-gray-200 text-gray-600 font-bold transition text-xs">-</button>
                                <span class="px-3 font-bold text-gray-800 text-xs">${item.stock_total}</span>
                                <button onclick="cambiarStock('${idCategoria}', '${idIdRecurso}', ${item.stock_total}, 1)" class="px-2 py-1 bg-gray-100 hover:bg-gray-200 text-gray-600 font-bold transition text-xs">+</button>
                            </div>
                        </div>
                    `;
                listaInventario.innerHTML += itemHTML;
            });
        });
    });
}

```

```

'${idIdRecurso}')" class="text-red-500 hover:text-red-700 p-2 transition ml-2">
    <i class="fa-solid fa-trash-can"></i>
  </button>
</div>
</div>
`;
listaInventario.insertAdjacentHTML('beforeend', itemHTML);
});
});
});
}

```

```

// NUEVA FUNCIÓN GLOBAL PARA ACTUALIZAR EL STOCK EN LA DB
window.cambiarStock = async function (categoria, idRecurso, stockActual, cambio) {
  const nuevoStock = stockActual + cambio;
  if (nuevoStock < 0) return; // Evitamos stock negativo

  try {
    const updates = {};
    updates[`inventario/${categoria}/${idRecurso}/stock_total`] = nuevoStock;
    await update(ref(db), updates);
    console.log(`Stock actualizado para ${idRecurso}: ${nuevoStock}`);
  } catch (error) {
    console.error("Error al actualizar stock:", error);
    alert("No se pudo actualizar el stock del equipo.");
  }
};

```

```

// ELIMINAR RECURSO
window.eliminarRecurso = async function (categoria, idRecurso) {
  const confirmar = confirm("¿Estás seguro de que querés eliminar este recurso del inventario de forma permanente? Se perderán sus datos de stock.");
  if (confirmar) {
    try {
      const recursoRef = ref(db, `inventario/${categoria}/${idRecurso}`);
      await remove(recursoRef);
      alert("Recurso eliminado del inventario con éxito.");
    } catch (error) {
      console.error("Error al eliminar:", error);
      alert("No se pudo eliminar el recurso.");
    }
  }
};

```

```

// =====
// MÓDULO B: CONTROL DE ADMINISTRADORES
// =====

```

```

// AGREGAR NUEVO ADMINISTRADOR
formNuevoAdmin.addEventListener('submit', async (e) => {

```

```

e.preventDefault();
const nombre = document.getElementById('admin-nombre').value.trim();
const email =
document.getElementById('admin-email').value.trim().toLowerCase();
const emailLimpio = email.replace(/\.\/g, '_');
try {
  const adminRef = ref(db, `administradores/${emailLimpio}`);
  await set(adminRef, {
    nombre: nombre,
    email: email
  });
  alert(`Se han asignado privilegios de administrador a ${nombre} de manera
exitosa.`);
  formNuevoAdmin.reset();
} catch (error) {
  console.error("Error al agregar administrador:", error);
  alert("Ocurrió un error al registrar el administrador en la base de
datos.");
}
});

```

```

// ESCUCHAR Y LISTAR ADMINISTRADORES

```

```

function escucharYListarAdmins() {
  const adminsRef = ref(db, 'administradores');
  onValue(adminsRef, (snapshot) => {
    listaAdministradores.innerHTML = '';
    const admins = snapshot.val();
    if (!admins) {
      listaAdministradores.innerHTML = '<p class="text-center text-gray-400
py-4">No hay administradores registrados.</p>';
      return;
    }
    Object.keys(admins).forEach(key => {
      const admin = admins[key];
      const esUsuarioPrincipal = admin.email === "laggerro2@gmail.com";
      const botonBorrar = esUsuarioPrincipal
        ? `<span class="text-xs bg-blue-100 text-blue-800 px-2.5 py-0.5
rounded-full font-semibold uppercase">Admin Principal</span>`
        : `<button onclick="eliminarAdmin('${key}', '${admin.nombre}')"
class="text-red-500 hover:text-red-700 p-2 transition">
          <i class="fa-solid fa-user-minus"></i>
        </button>`;
      const itemHTML = `
        <div class="flex items-center justify-between p-3 bg-gray-50 border
rounded-xl hover:shadow-sm transition-all mb-2">
          <div class="flex items-center gap-3">
            <div class="w-10 h-10 bg-indigo-100 rounded-full flex
items-center justify-center text-indigo-700">
              <i class="fa-solid fa-user-shield"></i>
            </div>

```

```

                <div>
                    <h4 class="font-bold text-gray-800 text-sm
leading-tight">${admin.nombre}</h4>
                    <p class="text-xs text-gray-500
font-medium">${admin.email}</p>
                </div>
            </div>
            ${botonBorrar}
        </div>
        `;
        listaAdministradores.insertAdjacentHTML('beforeend', itemHTML);
    });
});
}

// BORRAR ADMINISTRADOR
window.eliminarAdmin = async function (emailLimpio, nombreAdmin) {
    const confirmar = confirm(`¿Estás seguro de que querés retirarle los permisos
de administrador a "${nombreAdmin}"?\nEsta cuenta ya no podrá acceder a este panel
de ajustes ni ver el botón de configuración.`);
    if (confirmar) {
        try {
            const adminRef = ref(db, `administradores/${emailLimpio}`);
            await remove(adminRef);
            alert(`Permisos revocados para ${nombreAdmin} correctamente.`);
        } catch (error) {
            console.error("Error al revocar permisos:", error);
            alert("No se pudo dar de baja el administrador.");
        }
    }
};

// BOTÓN VOLVER
btnVolver.addEventListener('click', () => {
    window.location.href = "equipos.html";
});

// =====
// MÓDULO C: CONTROL DE INTENTOS Y PROFESORES
// =====

// 1. Escuchar la base de datos y llenar el desplegable
export function escucharYListarIntentosAcceso() {
    if (!selectIntentos) {
        console.error("No se encontró el elemento #select-intentos-acceso en el
DOM");
        return;
    }

    const intentosRef = ref(db, 'intentos_acceso');

```

```

onValue(intentosRef, (snapshot) => {
  selectIntentos.innerHTML = '<option value="">-- Seleccionar usuario no
autorizado --</option>';
  const intentos = snapshot.val();

  if (intentos) {
    Object.keys(intentos).forEach(key => {
      const item = intentos[key];
      const option = document.createElement('option');
      option.value = key;

      // Mapeo seguro según la estructura de Firebase
      const email = item.email || '';
      const nombre = item.nombre || item.email || 'Sin nombre';

      option.dataset.email = email;
      option.dataset.nombre = nombre;
      option.textContent = `${nombre} (${email})`;

      selectIntentos.appendChild(option);
    });
  }
}, (error) => {
  console.error("Error al leer intentos_acceso (Verificar Reglas de
Firebase):", error);
});
}

// 2. Autocompletar los inputs al seleccionar un mail del desplegable
if (selectIntentos) {
  selectIntentos.addEventListener('change', (e) => {
    const selectedOption = e.target.options[e.target.selectedIndex];
    if (selectedOption && selectedOption.value !== "") {
      inputNombre.value = selectedOption.dataset.nombre || '';
      inputEmail.value = selectedOption.dataset.email || '';
    } else {
      inputNombre.value = '';
      inputEmail.value = '';
    }
  });
}

// 3. Formulario para Autorizar y mover de "intentos_acceso" a
"usuarios_autorizados"
if (formNuevoProfesor) {
  formNuevoProfesor.addEventListener('submit', async (e) => {
    e.preventDefault();
    const nombre = inputNombre.value.trim();
    const email = inputEmail.value.trim().toLowerCase();
  });
}

```



```

const emailLimpio = email.replace(/\./g, '_');

try {
  // Guardar en usuarios_autorizados
  await set(ref(db, `usuarios_autorizados/${emailLimpio}`), {
    nombre: nombre,
    email: email,
    recibe_reporte: false
  });

  // Eliminar de intentos_acceso
  await remove(ref(db, `intentos_acceso/${emailLimpio}`));

  alert(`El docente ${nombre} ha sido autorizado correctamente.`);
  formNuevoProfesor.reset();
} catch (error) {
  console.error("Error al autorizar profesor:", error);
  alert("Error al guardar en la base de datos.");
}
});
}

// ESCUCHAR Y LISTAR PROFESORES
function escucharYListarProfesores() {
  const profesoresRef = ref(db, 'usuarios_autorizados');
  onValue(profesoresRef, (snapshot) => {
    listaProfesores.innerHTML = '';
    const profesores = snapshot.val();
    if (!profesores) {
      listaProfesores.innerHTML = '<p class="text-center text-gray-400 py-4">No hay docentes autorizados en la lista.</p>';
      return;
    }
    Object.keys(profesores).forEach(key => {
      const profe = profesores[key];
      const esPropietario = profe.email === "laggerro2@gmail.com";
      const botonBorrar = esPropietario
        ? `<span class="text-xs bg-blue-100 text-blue-800 px-2.5 py-0.5 rounded-full font-semibold">Admin Principal</span>`
        : `<button onclick="eliminarProfesor('${key}', '${profe.nombre}')"`
          + `class="text-red-500 hover:text-red-700 p-2 transition">`
          + `<i class="fa-solid fa-user-slash"></i>`
          + `</button>`;

      const itemHTML = `
        <div class="flex items-center justify-between p-3 bg-gray-50 border rounded-xl hover:shadow-sm transition-all mb-2">
          <div class="flex items-center gap-3">
            <div class="w-10 h-10 bg-green-100 rounded-full flex items-center justify-center text-green-700">

```

```

        <i class="fa-solid fa-chalkboard-user"></i>
      </div>
      <div>
        <h4 class="font-bold text-gray-800 text-sm
leading-tight">${profe.nombre}</h4>
        <p class="text-xs text-gray-500
font-medium">${profe.email}</p>
        <label class="inline-flex items-center gap-1.5 mt-1
cursor-pointer select-none">
          <input type="checkbox" id="check-rep-${key}"
class="w-3.5 h-3.5 rounded text-green-600 focus:ring-green-500 cursor-pointer"
${profe.recibe_reporte ? 'checked' : ''}>
          <span class="text-[10px] text-gray-500 font-bold
uppercase tracking-wider">¿Recibe Reporte Diario?</span>
        </label>
      </div>
    </div>
    ${botonBorrar}
  </div>
  `;
  listaProfesores.insertAdjacentHTML('beforeend', itemHTML);

  const checkReporte = document.getElementById(`check-rep-${key}`);
  if (checkReporte) {
    checkReporte.addEventListener('change', async (e) => {
      const nuevoEstado = e.target.checked;
      try {
        const updates = {};
        updates[`usuarios_autorizados/${key}/recibe_reporte`] =
nuevoEstado;

        await update(ref(db), updates);
        console.log(`Preferencia de reporte actualizada para
${profe.nombre}: ${nuevoEstado}`);
      } catch (err) {
        console.error("Error al actualizar preferencia de
reporte:", err);

        alert("No se pudo actualizar la configuración.");
        e.target.checked = !nuevoEstado; // Revierte el check
      }
    });
  }
});
});
});
}

// QUITAR PROFESOR DE LA LISTA BLANCA
window.eliminarProfesor = async function (emailLimpio, nombreProfe) {
  const confirmar = confirm(`¿Estás seguro de que querés REVOCAR el acceso a
"${nombreProfe}"?\nSi lo hacés, perderá la capacidad de iniciar sesión de forma
inmediata.`);

```

```

    if (confirmar) {
      try {
        const profeRef = ref(db, `usuarios_autorizados/${emailLimpio}`);
        await remove(profeRef);
        const adminRef = ref(db, `administradores/${emailLimpio}`);
        await remove(adminRef);
        alert(`Acceso revocado con éxito para ${nombreProfe}.`);
      } catch (error) {
        console.error("Error al revocar acceso:", error);
        alert("No se pudo completar la operación.");
      }
    }
  });
  // Borrar la lista completa de intentos de acceso
  if (btnBorrarIntentos) {
    btnBorrarIntentos.addEventListener('click', async () => {
      const confirmacion = confirm("¿Estás seguro de que querés borrar TODAS las solicitudes de acceso pendientes?\nEsta acción no se puede deshacer.");
      if (confirmacion) {
        try {
          await remove(ref(db, 'intentos_acceso'));
          alert("La lista de solicitudes pendientes ha sido eliminada.");
          inputNombre.value = '';
          inputEmail.value = '';
        } catch (error) {
          console.error("Error al borrar solicitudes pendientes:", error);
          alert("Ocurrió un error al intentar borrar la lista.");
        }
      }
    });
  }
}

// =====
// MÓDULO D: CONFIGURACIÓN GENERAL DEL REPORTE
// =====

// 1. Leer la hora guardada en Firebase y rellenar el input
async function cargarConfiguracionHora() {
  try {
    const configRef = ref(db, 'configuracion/hora_reporte');
    const snapshot = await get(configRef);

    if (snapshot.exists()) {
      configHoraReporte.value = snapshot.val();
      console.log("Hora de reporte cargada desde la BD:", snapshot.val());
    } else {
      configHoraReporte.value = "08:00"; // Hora por defecto
      console.log("No se encontró hora configurada. Se estableció 08:00 por defecto.");
    }
  }
}

```

```

    }
  } catch (error) {
    console.error("Error al cargar la hora del reporte:", error);
  }
}

// 2. Guardar la nueva hora seleccionada por el Administrador
btnGuardarHora.addEventListener('click', async (e) => {
  e.preventDefault(); // Evita recargas
  const nuevaHora = configHoraReporte.value.trim();
  if (!nuevaHora) {
    alert("Por favor, ingresá al menos un horario válido.");
    return;
  }

  try {
    btnGuardarHora.disabled = true;
    btnGuardarHora.innerHTML = `

```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Panel de Control - ISD</title>
  <link rel="icon" type="image/png" href="favicon.png" sizes="32x32">
  <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.0.0/css/all.min.css">
</head>

<body class="bg-gray-100 min-h-screen">

  <nav class="bg-blue-800 text-white p-4 flex justify-between items-center
shadow-md">
    <h1 class="text-xl font-bold"><i class="fa-solid fa-gears mr-2"></i>Ajustes
del Sistema</h1>
    <button id="btn-volver"
      class="bg-blue-600 hover:bg-blue-700 text-white px-4 py-2 rounded
text-sm font-semibold transition">
      ← Volver a Equipos
    </button>
  </nav>

  <!-- CONTENEDOR PRINCIPAL -->
  <main class="max-w-7xl mx-auto my-8 p-4 grid grid-cols-1 lg:grid-cols-2 gap-8">

    <!-- SECCIÓN DE EQUIPOS (INVENTARIO) -->
    <section class="space-y-6">
      <div class="bg-white p-6 rounded-2xl shadow-md">
        <h2 class="text-lg font-bold text-gray-800 mb-4 border-b pb-2">
          <i class="fa-solid fa-list text-blue-600 mr-1"></i>Equipos
Registrados
        </h2>
        <div id="lista-inventario" class="space-y-4 max-h-[450px]
overflow-y-auto pr-2">
          <p class="text-center text-gray-500 py-4">Cargando
inventario...</p>
        </div>
      </div>

      <!-- NUEVO: CONFIGURACIÓN GENERAL DEL SISTEMA (Habilitar y programar
reporte) -->
      <div class="bg-white p-6 rounded-2xl shadow-md border-t-4
border-amber-500 mt-6">
        <h2 class="text-lg font-bold text-gray-800 mb-4 border-b pb-2">
          <i class="fa-solid fa-clock text-amber-600 mr-1"></i>
Configuración de Reporte Diario
        </h2>
        <p class="text-xs text-gray-500 mb-4">
          Definí a qué hora se enviará el correo resumen.

```

```

        <strong>Podés ingresar varias horas separadas por coma (ej:
08:00, 13:30, 19:00)</strong>.
    </p>
    <div class="flex flex-col sm:flex-row items-end gap-4">
        <div class="flex-grow">
            <label class="block text-gray-700 text-xs font-bold
mb-1">Horarios de Envío (Formatos de 24
                hs):</label>
            <div class="relative">
                <span class="absolute inset-y-0 left-0 flex
items-center pl-3 text-gray-400">
                    <i class="fa-regular fa-clock text-xs"></i>
                </span>
                <!-- Cambiamos el input de "time" a "text" para que
permite escribir comas y múltiples horas de corrido -->
                <input type="text" id="config-hora-reporte"
                    class="w-full border rounded-xl pl-9 pr-3 py-2
text-sm focus:ring-2 focus:ring-amber-500 focus:outline-none"
                    placeholder="Ej: 08:00, 13:00, 19:30" required>
            </div>
        </div>
        <button id="btn-guardar-hora"
            class="w-full sm:w-auto bg-amber-600 hover:bg-amber-700
text-white font-bold py-2.5 px-5 rounded-xl shadow-md text-xs transition-all
active:scale-98 flex items-center justify-center gap-2 whitespace-nowrap">
            <i class="fa-solid fa-floppy-disk"></i> Guardar Horarios
        </button>
    </div>
</div>
<div class="bg-white p-6 rounded-2xl shadow-md">
    <h2 class="text-lg font-bold text-gray-800 mb-4 border-b pb-2">
        <i class="fa-solid fa-plus-circle text-blue-600 mr-1"></i>Nuevo
Recurso
    </h2>

    <form id="form-nuevo-recurso" enctype="multipart/form-data">
        <div class="mb-4">
            <label class="block text-gray-700 text-xs font-bold
mb-1">Categoría:</label>
            <select id="recurso-categoria"
                class="w-full border rounded-xl px-3 py-2 text-sm
focus:ring-2 focus:ring-blue-500"
                required>
                <option value="" disabled selected>Elegí
categoría...</option>
                <option value="tecnologia">Tecnología (Notebooks,
Impresoras 3D)</option>
                <option value="audiovisual">Audiovisual (Proyectores,
Audio)</option>
                <option value="didacticos">Didácticos (Robótica,

```

Mapas)</option>

</select>

</div>

<div class="mb-4">

<label class="block text-gray-700 text-xs font-bold
mb-1">Nombre del Recurso:</label>

<input type="text" id="recurso-nombre" class="w-full border
rounded-xl px-3 py-2 text-sm"
placeholder="Ej: Proyector Epson HDMI" required>

</div>

<div class="mb-4">

<label class="block text-gray-700 text-xs font-bold
mb-1">Cantidad (Stock):</label>

<input type="number" id="recurso-cantidad" min="1"
class="w-full border rounded-xl px-3 py-2 text-sm"
placeholder="Ej: 4" required>

</div>

<div class="mb-4">

<label class="block text-gray-700 text-xs font-bold
mb-1">Descripción:</label>

<textarea id="recurso-descripcion" rows="3" class="w-full
border rounded-xl px-3 py-2 text-sm"
placeholder="Breve explicación de uso o accesorios que
incluye..."></textarea>
</div>

<div class="mb-6">

<label class="block text-gray-700 text-xs font-bold
mb-1">Imagen (Foto del Recurso):</label>

<input type="file" id="recurso-imagen" accept="image/*"
class="w-full text-xs text-gray-500 file:mr-4 file:py-2
file:px-4 file:rounded-xl file:border-0 file:text-xs file:font-semibold
file:bg-blue-50 file:text-blue-700 hover:file:bg-blue-100"
required>

</div>

<button type="submit"

class="w-full bg-blue-600 hover:bg-blue-700 text-white
font-bold py-3 rounded-xl shadow-md text-sm transition-all active:scale-98">
Guardar en Inventario

</button>

</form>

</div>

</section>

<!-- SECCIÓN DE ADMINISTRADORES / AUXILIARES (NUEVA) -->

```

<!-- SECCIÓN DE USUARIOS Y PERMISOS (LADO DERECHO) -->
<section class="space-y-6">

    <!-- FORMULARIO 1: AUTORIZAR PROFESORES (NUEVO) -->
    <div class="bg-white p-6 rounded-2xl shadow-md">
        <h2 class="text-lg font-bold text-gray-800 mb-4 border-b pb-2">
            <i class="fa-solid fa-user-plus text-green-600
mr-1"></i>Autorizar Nuevo Docente
        </h2>
        <p class="text-xs text-gray-500 mb-4">Los usuarios podrán iniciar
sesión y reservar recursos, pero no
            tendrán acceso a este panel de ajustes.</p>

        <form id="form-nuevo-profesor">
            <div class="mb-4">
                <label class="block text-gray-700 text-xs font-bold mb-1">
                    <i class="fa-solid fa-user-clock text-amber-500
mr-1"></i>Solicitudes de Acceso Pendientes:
                </label>
                <select id="select-intentos-acceso"
                    class="w-full border rounded-xl px-3 py-2 text-sm
bg-amber-50 border-amber-300 focus:ring-2 focus:ring-amber-500">
                    <option value="">-- Seleccionar usuario no autorizado
--</option>
                </select>
            </div>

            <div class="mb-4">
                <label class="block text-gray-700 text-xs font-bold
mb-1">Nombre del Docente:</label>
                <input type="text" id="profe-nombre" class="w-full border
rounded-xl px-3 py-2 text-sm"
                    placeholder="Ej: Prof. María Giménez" required>
            </div>
            <div class="mb-6">
                <label class="block text-gray-700 text-xs font-bold
mb-1">Correo Electrónico:</label>
                <input type="email" id="profe-email" class="w-full border
rounded-xl px-3 py-2 text-sm"
                    placeholder="Ej: m.gimenez@colegio.edu" required>
            </div>
            <div class="flex flex-col sm:flex-row gap-3">
                <button type="submit"
                    class="w-full sm:w-1/2 bg-green-600 hover:bg-green-700
text-white font-bold py-3 rounded-xl shadow-md text-sm transition-all
active:scale-98">
                    <i class="fa-solid fa-user-check mr-1"></i> Autorizar
Docente
                </button>

```



```

        <button type="button" id="btn-borrar-intentos"
            class="w-full sm:w-1/2 bg-red-600 hover:bg-red-700
text-white font-bold py-3 rounded-xl shadow-md text-sm transition-all
active:scale-98">
            <i class="fa-solid fa-trash-can mr-1"></i> Limpiar
Solicitudes
        </button>
    </div>
</form>
</div>

<!-- TABLA/LISTA: PROFESORES AUTORIZADOS (NUEVO) -->
<div class="bg-white p-6 rounded-2xl shadow-md">
    <h2 class="text-lg font-bold text-gray-800 mb-4 border-b pb-2">
        <i class="fa-solid fa-users text-green-600 mr-1"></i>Profesores
Autorizados
    </h2>
    <div id="lista-profesores" class="space-y-4 max-h-[250px]
overflow-y-auto pr-2">
        <p class="text-center text-gray-500 py-4">Cargando lista de
profesores...</p>
    </div>
</div>

<!-- FORMULARIO 2: AGREGAR ADMINS / AUXILIARES -->
<div class="bg-white p-6 rounded-2xl shadow-md border-t-4
border-indigo-500">
    <h2 class="text-lg font-bold text-gray-800 mb-4 border-b pb-2">
        <i class="fa-solid fa-user-shield text-indigo-600
mr-1"></i>Otorgar Rol de Administrador
    </h2>
    <p class="text-xs text-gray-500 mb-4"><strong>Nota:</strong> Para
ser administrador, el usuario ya debe
        estar registrado como Docente Autorizado arriba.</p>

    <form id="form-nuevo-admin">
        <div class="mb-4">
            <label class="block text-gray-700 text-xs font-bold
mb-1">Nombre Completo:</label>
            <input type="text" id="admin-nombre" class="w-full border
rounded-xl px-3 py-2 text-sm"
                placeholder="Ej: Juan Pérez" required>
        </div>
        <div class="mb-6">
            <label class="block text-gray-700 text-xs font-bold
mb-1">Correo Electrónico:</label>
            <input type="email" id="admin-email" class="w-full border
rounded-xl px-3 py-2 text-sm"
                placeholder="Ej: auxiliar@colegio.edu" required>
        </div>
    </form>

```

```

        <button type="submit"
            class="w-full bg-indigo-600 hover:bg-indigo-700 text-white
font-bold py-3 rounded-xl shadow-md text-sm transition-all active:scale-98">
            Dar Permisos de Admin
        </button>
    </form>
</div>

<!-- TABLA/LISTA: ADMINISTRADORES -->
<div class="bg-white p-6 rounded-2xl shadow-md">
    <h2 class="text-lg font-bold text-gray-800 mb-4 border-b pb-2">
        <i class="fa-solid fa-users-gear text-indigo-600
mr-1"></i>Lista de Administradores
    </h2>
    <div id="lista-administradores" class="space-y-4 max-h-[200px]
overflow-y-auto pr-2">
        <p class="text-center text-gray-500 py-4">Cargando
administradores...</p>
    </div>
</div>

</section>

</main>
<footer
    class="fixed bottom-0 left-0 w-full bg-gray-300 text-black text-center py-2
text-xs font-semibold shadow-inner z-50">
    by Lagersoft - 2026
</footer>
<script type="module" src="ajustes.js"></script>
</body>

</html>

```